SEV 펌웨어 관리

개요

AMD SEV (Secure Encrypted Virtualization) 펌웨어는 AMD EPYC 프로세서의 Platform Security Processor (PSP) 내에서 실행되는 핵심 소프트웨어입니다. 이 펌웨어는 보안 가상화 기능을 제공하고, 호스트와 게스트 간의 신뢰할 수 있는 실행 환경을 구축하는 역할을 담당합니다.

SEV 펌웨어의 역할과 기능

주요 역할

1. **플랫폼 보안 관리**: SEV 플랫폼의 전반적인 보안 상태를 관리하고 모니터링
2. **암호화 키 관리**: 플랫폼 암호화 키(PEK), 플랫폼 Diffie-Hellman 키(PDH) 등의 생성 및 관리
3. **게스트 VM 보안**: 게스트 가상 머신의 메모리 암호화 및 무결성 보장
4. **인증서 체인 관리**: 신뢰 체인 구축을 위한 인증서 생성, 검증, 관리
5. **원격 증명**: 게스트 VM의 상태를 원격으로 증명할 수 있는 기능 제공

핵심 기능

1. 플랫폼 상태 관리

- **Uninitialized**: 초기 상태, 아직 설정되지 않음
- **Initialized**: 기본 설정 완료, 게스트 실행 준비 중
- **Working**: 정상 작동 상태, 게스트 VM 실행 가능

2. 키 생성 및 관리

- **PEK (Platform Encryption Key)**: 플랫폼 암호화의 기본 키
- **PDH (Platform Diffie-Hellman)**: 키 교환을 위한 공개키 쌍
- **CEK (Chip Endorsement Key)**: 칩 수준의 인증 키

3. 인증서 처리

- 인증서 체인 검증 (ARK → ASK → CEK → PEK)
- 인증서 내보내기 및 가져오기
- 신뢰 체인 구축

펌웨어 인터페이스 사용법

펌웨어 핸들 생성

```
use sev::firmware::host::Firmware;

// SEV 플랫폼에 대한 핸들 생성
let mut firmware = Firmware::open()?;
```

플랫폼 상태 확인

```
use sev::firmware::host::{Firmware, Status, State};

let mut firmware = Firmware::open()?;

// 플랫폼 상태 조회
```

```rust
let status: Status = firmware.platform_status()?;

println!("펌웨어 버전: {}.{}",
status.build.version.major,
status.build.version.minor);
println!("빌드 번호: {}", status.build.build);
println!("활성 게스트 수: {}", status.guests);

match status.state {
State::Uninitialized => println!("플랫폼 상태: 초기화되지 않음"),
State::Initialized => println!("플랫폼 상태: 초기화 완료"),
State::Working => println!("플랫폼 상태: 작동 중"),
}
```

플랫폼 식별자 조회

```rust
use sev::firmware::host::{Firmware, Identifier};

let mut firmware = Firmware::open()?;

// CPU 고유 식별자 조회
let identifier: Identifier = firmware.get_identifier()?;
println!("플랫폼 식별자: {}", identifier);
```

```
// 바이트 배열로 변환
let id_bytes: Vec<u8> = identifier.into();
```

펌웨어 업데이트 프로세스

1. 현재 펌웨어 버전 확인

펌웨어 업데이트 전에 현재 설치된 펌웨어 버전을 확인해야 합니다:

```
use sev::firmware::host::{Firmware, Build, Version};

let mut firmware = Firmware::open()?;
let status = firmware.platform_status()?;

// 최소 요구 버전 확인
let minimum_version = Build {
version: Version { major: 0, minor: 14 },
build: 0,
};

if status.build > minimum_version {
println!("펌웨어 버전이 요구사항을 만족합니다.");
} else {
```

```
println!("펌웨어 업데이트가 필요합니다.");
}
```

2. 펌웨어 업데이트 절차

펌웨어 업데이트는 일반적으로 다음 단계를 따릅니다:

1. **시스템 준비**: 모든 게스트 VM 종료
2. **플랫폼 리셋**: 기존 상태 초기화
3. **새 펌웨어 설치**: BIOS/UEFI를 통한 펌웨어 업데이트
4. **시스템 재부팅**: 새 펌웨어 활성화
5. **상태 검증**: 업데이트 성공 여부 확인

3. 업데이트 후 검증

```
use sev::firmware::host::Firmware;

let mut firmware = Firmware::open()?;

// 업데이트 후 상태 확인
let status = firmware.platform_status()?;
println!("업데이트된 펌웨어 버전: {}.{}",
status.build.version.major,
status.build.version.minor);
```

```
// 플랫폼 기능 테스트
let identifier = firmware.get_identifier()?;
println!("플랫폼 식별자 조회 성공: {}", identifier);
```

펌웨어 상태 관리 및 모니터링

상태 모니터링 시스템

```
use sev::firmware::host::{Firmware, State};
use std::time::{Duration, Instant};

struct FirmwareMonitor {
firmware: Firmware,
last_check: Instant,
check_interval: Duration,
}

impl FirmwareMonitor {
fn new() -> Result<Self, Box<dyn std::error::Error>> {
Ok(FirmwareMonitor {
firmware: Firmware::open()?,
last_check: Instant::now(),
check_interval: Duration::from_secs(30), // 30초마다 확인
```

```
})
}

fn check_status(&mut self) -> Result<(), Box<dyn std::error::Error>> {
if self.last_check.elapsed() >= self.check_interval {
let status = self.firmware.platform_status()?;
match status.state {
State::Working => {
println!("플랫폼 정상 작동 중 - 게스트 수: {}", status.guests);
}
State::Initialized => {
println!("플랫폼 초기화 완료 - 게스트 대기 중");
}
State::Uninitialized => {
println!("경고: 플랫폼이 초기화되지 않음");
}
}
self.last_check = Instant::now();
}
Ok(())
}
}
```

오류 상황 처리

```
use sev::firmware::host::Firmware;
use sev::error::{UserApiError, SevError};

fn handle_firmware_errors(mut firmware: Firmware) {
match firmware.platform_status() {
Ok(status) => {
println!("플랫폼 상태 조회 성공");
}
Err(UserApiError::SevError(SevError::InvalidPlatformState)) => {
println!("오류: 잘못된 플랫폼 상태");
// 플랫폼 리셋 시도
if let Err(e) = firmware.platform_reset() {
println!("플랫폼 리셋 실패: {:?}", e);
}
}
Err(UserApiError::IOError(io_err)) => {
println!("I/O 오류: {}", io_err);
// /dev/sev 장치 접근 권한 확인 필요
}
Err(e) => {
println!("기타 오류: {:?}", e);
```

```
}

}

}
```

SEV-SNP 펌웨어 관리

SNP 플랫폼 상태 조회

```rust
use sev::firmware::host::{Firmware, SnpPlatformStatus};

let mut firmware = Firmware::open()?;

// SEV-SNP 플랫폼 상태 조회
let snp_status: SnpPlatformStatus = firmware.snp_platform_status()?;

println!("SNP 펌웨어 버전: {}.{}", snp_status.version.0, snp_status.version.1);
println!("빌드 ID: {}", snp_status.build_id);
println!("활성 게스트 수: {}", snp_status.guest_count);
println!("플랫폼 TCB 버전: {:?}", snp_status.platform_tcb_version);
println!("보고된 TCB 버전: {:?}", snp_status.reported_tcb_version);
```

SNP 설정 관리

```rust
use sev::firmware::host::{Firmware, Config, TcbVersion, MaskId};
```

```rust
let mut firmware = Firmware::open()?;

// SNP 설정 생성
let config = Config::new(
TcbVersion::new(Some(3), 0, 10, 169, 0), // TCB 버전
MaskId(0), // 마스크 ID
);

// 설정 적용
firmware.snp_set_config(config)?;
println!("SNP 설정이 성공적으로 적용되었습니다.");
```

SNP 커밋 작업

```rust
use sev::firmware::host::Firmware;

let mut firmware = Firmware::open()?;

// SNP 커밋 실행
// 이 작업은 다음을 수행합니다:
// - CommittedTCB를 CurrentTCB로 설정
// - CommittedVersion을 FirmwareVersion으로 설정
```

```
// - ReportedTCB를 CurrentTCB로 설정

// - ReportedTCB가 변경된 경우 VLEK hashstick 삭제

firmware.snp_commit()?;

println!("SNP 커밋이 성공적으로 완료되었습니다.");
```

보안 고려사항

1. 권한 관리

- SEV 펌웨어 인터페이스는 root 권한이 필요합니다
- `/dev/sev` 장치에 대한 적절한 접근 권한 설정이 필요합니다

2. 상태 검증

- 펌웨어 작업 전후에 항상 상태를 검증해야 합니다
- 예상치 못한 상태 변화에 대한 적절한 오류 처리가 필요합니다

3. 동시성 제어

- 여러 프로세스가 동시에 펌웨어에 접근하지 않도록 주의해야 합니다
- 필요한 경우 적절한 락킹 메커니즘을 구현해야 합니다

문제 해결

일반적인 오류 상황

1. **장치 접근 오류**

오류: Permission denied (os error 13)

해결: sudo 권한으로 실행하거나 사용자를 적절한 그룹에 추가

2. **플랫폼 상태 오류**

오류: Invalid platform state

해결: platform_reset() 호출 후 재시도

3. **펌웨어 버전 호환성**

오류: Unsupported firmware version

해결: 펌웨어 업데이트 필요

디버깅 팁

```rust
use sev::firmware::host::Firmware;


// 디버깅을 위한 상세 정보 출력
fn debug_firmware_state() -> Result<(), Box<dyn std::error::Error>> {
let mut firmware = Firmware::open()?;
// 기본 상태 정보
let status = firmware.platform_status()?;
println!("=== 펌웨어 상태 디버그 정보 ===");
println!("버전: {}.{}", status.build.version.major, status.build.version.minor);
```

```
println!("빌드: {}", status.build.build);

println!("상태: {:?}", status.state);

println!("게스트 수: {}", status.guests);

println!("플래그: {:?}", status.flags);

// 플랫폼 식별자

let  identifier = firmware.get_identifier()?;

println!("식별자: {}", identifier);

Ok(())

}
```

이 문서는 SEV 펌웨어 관리의 기본적인 개념과 실제 구현 방법을 제공합니다. 실제 운영 환경에서는 보안 정책과 시스템 요구사항에 따라 추가적인 고려사항이 필요할 수 있습니다.